v2 Study: Rebuilding the Research Behind the 7-Page Paper

Overview

Rebuild the evaluation pipeline and experiments so that every result in the paper comes from a model a hardware reviewer would accept, then rewrite the 7-page IEEE conference paper around the new results. Changes, in short:

- bit-width affects accuracy through real whole-network quantization;
- the scheduler shares the core with periodic RTOS tasks, so τ and π matter;
- energy, cycles and memory are calibrated to the STM32F446RE;
- memory becomes two hard constraints;
- only the multi-objective problem remains;
- six search methods are compared, including a joint-vs-sequential ablation, over 20–30 seeds;
- latency and memory predictions are checked against real STM32 boards through STM32Cube.AI Developer Cloud;
- every design choice is backed by a journal article printed in 2020 or later, or is labelled as an assumption and covered by a sensitivity check (`docs/plans/2026-10-11-v2-literature-audit.md`);
- the new code is organised as five deep modules behind one entry point, `python -m src.pipeline <stage>`, instead of a flat `src/` of about 22 files (see Code Layout below).

All compute runs on a fanless M4 MacBook Air (16 GB) in resumable chunks of at most 60 minutes.

Current State Analysis (v1, commit `5176d5f`)

Area	v1 behavior	Problem
Accuracy	3-parameter log fit to 6 architectures (`src/objective.py _fit_scaling_law`)	Flat in b; too few points; checkpoint chosen on the test set (`src/train.py:98–108`)
Quantization	`quantize_dynamic` on `nn.Linear` only; 4-bit = 8-bit with halved size (`src/quantization.py`)	Conv weights never quantized; 4-bit never executes
Energy	$F \cdot V^2 \cdot C \cdot \delta \cdot b/32$ (`src/energy_proxy.py`)	No static/sleep power; b/32 invented; V, C assumed
Scheduler	one Poisson stream, identical priority (`src/scheduler_sim.py`)	π=1 ≡ π=2; π=0 is FCFS; no context-switch cost
Memory	reported, never constrained	512 KB assumption, no SRAM model
Search	SOO + MOO; baselines Random and Grid only	No joint-vs-sequential test; no NSGA-II/TPE
Seeds	MOO headline 1 seed × 100; sweep 15 × 20	Inconsistent budgets
Paper	7 pages, 2 TikZ diagrams, no result plots	

Measured cost: one 50-epoch training ≈ 25 min on MPS (≈30 s/epoch, `logs/training.log`). The training loop runs a second forward pass per batch only to compute training accuracy (`src/train.py`, `_, pred = model(images).max(1)`). qNEHVI ≈ 10–15 min per 100-trial seed (`config/config.py` comments).

Environment: `.venv` with Python 3.14.7, Ax 1.3.1, BoTorch 0.18.1, pymoo 0.6.1.6, SimPy 4.1.2. Optuna is not installed.

Desired End State

- Branch `v2-study` holds the v2 code, results in `results/v2/`, figures in `figures/v2/`, and the rewritten paper in `paper/` that compiles to exactly 7 filled pages.
- Every number in the paper is emitted by `src/report/paper_numbers.py` into `results/v2/paper_numbers.json`, and a check script confirms that each number in the `.tex` files appears there.
- `pytest` passes for the job runner, results store, network, evaluator and search harness, each tested through its own interface.
- `src/` holds only the Code Layout below; every v1 module has been deleted or replaced (v1 code stays in git history).
- `python -m src.pipeline status` lists every stage, its outputs and whether it is done.
- The style memory rules hold: banned words, table format, sci notation, grayscale figures, journal-only references.

Key Decisions (from the 2026-10-11 interview)

1. Format: 7-page IEEE conference paper (CONECCT 2027 / VLSID 2028).
2. Bit-width affects accuracy through real quantization of every layer.
3. Workload: inference shares the core with periodic tasks under FreeRTOS-style fixed-priority preemption.
4. Energy, cycle and memory models calibrated to the STM32F446RE.
5. Flash ≤ 512 KB and SRAM ≤ 128 KB as hard constraints.
6. Design space: $L \in \{2,3,4,5\}$, $w \in \{0.25, 0.4375, 0.625, 0.8125, 1.0\}$ (discrete, ordered), $b \in \{4,8,16,32\}$, $\tau \in [1,20]$ ms, $\pi \in$ {below, equal, above}, $f \in$ five datasheet operating points.
7. Accuracy: measured table over the 20-architecture grid × 4 b (tabular, as in NATS-Bench; no interpolation), plus 3×2 extra seeds for training noise. *(Changed by the literature audit: PCHIP interpolation had no journal precedent.)*
8. The single-objective problem is dropped.
9. Methods: qNEHVI (proposed), qNParEGO, NSGA-II, MOTPE, Random, and the Sequential ablation; Grid is dropped. 100 evaluations each; 30 seeds if the Phase 6 pilot allows, otherwise 20.
10. Workload parameters and thresholds as in Phase 3 and Phase 5.
11. Hardware validation through STM32Cube.AI Developer Cloud.
12. References remain journal-only **and printed in 2020 or later**. Datasheets and manuals are named in the text, not cited. Every plan item's backing is listed in `docs/plans/2026-10-11-v2-literature-audit.md`.
13. Figures: redrawn architecture diagram, front plot and convergence plot. Predicted-vs-measured cycles as a figure only if space allows, otherwise a table.
14. Compute: chunks of at most 60 min, `nice`, 4 torch threads, foreground only, started only on request.
15. Branch `v2-study`; v1 outputs archived to `results/v1/` and `figures/v1/`.
16. Code layout: five deep modules (job runner, results store, network, evaluator, search harness) plus `report/`, driven by one `src/pipeline.py`. Each phase writes its files straight into this layout, so there is no separate refactor step (from the 2026-10-11 architecture review).

What We're NOT Doing

- No 2-bit precision, no second dataset, no change of network family beyond adding L=5.
- No owned hardware and no current measurement. Energy stays datasheet-calibrated; only cycles and memory are checked on silicon.
- No multi-core target, no DMA or cache modeling beyond the flash-read term.
- No single-objective problem, no grid-search baseline.
- No conference, preprint or datasheet entries in `refs.bib`.
- No change of page count, title or authors.
- No overnight, multi-hour or background compute.
- No rewriting of paper prose before the Phase 6 results are final.

Implementation Approach

Build bottom-up, with each model tested in isolation before anything depends on it:

1. hardware cost models;
2. calibration against ST silicon;
3. the workload simulator;
4. accuracy data;
5. the unified `evaluate()` and the reference front;
6. searches;
7. held-out hardware validation;
8. figures;
9. the paper.

Cycle calibration needs only architectures, not trained weights, so it happens early (Phase 2), before training. The final hardware check uses the searches' front designs as held-out data (Phase 7).

Code Layout

```
src/
  pipeline.py        # the only entry point: python -m src.pipeline <stage> [options] | status
  jobs.py            # Job runner: resume, 55-min deadline, skip-done, atomic writes   (Phase 0)
  results.py         # Results store: every artifact by name, path and producing stage   (Phase 0)
  network/           # Arch spec → profile, torch model, ONNX; train; quantize           (Phases 0, 2, 4)
  evaluator/         # evaluate(design, scenario) → Outcome; Scenario; cost models,
                     #   RTOS simulator, accuracy table, calibration                     (Phases 1–5)
  search/            # space, harness, 6 method adapters, reference front, stats         (Phases 5–6)
  report/            # validation summaries, figures, tables, paper_numbers             (Phases 4, 7–9)
config/config.py     # paths and compute settings only; no science constants
tests/               # one file per deep module, through its interface
scripts/check_paper_numbers.py
```

Rules every phase follows:

- **No reaching past an interface.** A module imports another module's public names only (v1 broke this: `src/reference_front.py` imports the private `_arch_model_and_stats` from `src/objective.py`).
- **Stages are short.** Each stage in `pipeline.py` combines the deep modules in a few lines: it builds a job list, calls `run_jobs`, and saves through `results`. Stage logic that grows beyond that belongs inside a module.

- **No result paths outside** `results.py`**.** Other code calls `results.load(name)` / `results.save(name, data)`.
- **No module-level science constants.** Device, workload, kernel costs, idle mode, limits and calibration live in a frozen `Scenario` value passed to `evaluate`.
- **No import-time side effects.** `nice` and thread limits are set when `run_jobs` starts, not at import.
- **Tests go through the interface.** A test that needs to reach past an interface means the module has the wrong shape; fix the shape, not the test.
- **v1 modules are deleted in the phase that replaces them.** None are kept alongside v2 code.

Chunk accounting (estimates, re-measured in Phase 0):

Work	Chunks (≤60 min)
Training benchmark + speedups	done (20 s; 5.5 s/epoch worst case, 50 epochs ≈ 4.6 min)
QAT epoch pilot (2 architectures)	1
Training: 26 runs + QAT + quantized evaluation	2–3 (re-estimated from the measured 5.5 s/epoch; QAT time still unmeasured)
Reference front + sensitivity	1–2
Searches: 6 methods × 20 seeds (30 if pilot allows)	14–16 (21–24)
Everything else (tests, plots, export)	light, within normal sessions

---

Phase 0: Setup and Training Speedups

Overview

Branch, archive v1, install tools, make training fast and resumable, and create the validation split.

Changes Required

1. Branch and archive

- `git switch -c v2-study` from `paper-full-draft`.
- `git mv` the current `results/*` into `results/v1/` and `figures/*` into `figures/v1/`. Keep `results/v1/` read-only by convention.
- `config/config.py`: `RESULTS_DIR = PROJECT_ROOT / "results" / "v2"` and `FIGURES_DIR = PROJECT_ROOT / "figures" / "v2"`.

2. Tools

- `.venv/bin/pip install optuna pytest onnx onnxruntime`. If either ONNX package lacks a Python 3.14 wheel, create a second env with `uv venv --python 3.12 .venv-export` used only for model export.
- Add the new pins to `requirements.txt`.

3. Job runner (replaces the planned `compute_guard.py`)

New file `src/jobs.py`, interface `run_jobs(stage, jobs, work, minutes=55)`:

- a job is a key plus the results-store artifact (or file) it produces; a job is done when its output exists, so a rerun skips it;
- on entry: `torch.set_num_threads(4)` and `os.nice(10)` (not at import);
- the deadline is checked between jobs, and `work` receives a `deadline` it can check between finer units (epochs);
- outputs are written to a temp file and renamed, so a killed run never leaves a half-written CSV;
- prints elapsed time, finished jobs and remaining jobs on exit. Every chunked stage in later phases (training, accuracy collection, reference-front blocks, method × seed runs) uses this module; none writes its own resume or deadline logic.

4. Results store

New file `src/results.py`:

- a registry of named artifacts, each with path (under `RESULTS_DIR`), format and producing stage, for example `"accuracy_measurements"`, `"cycle_calibration"`, `"reference_front"`, `"run/<method>/<seed>"`, `"comparison_summary"`;
- `load(name)` and `save(name, data)`; loading a missing artifact raises "run stage `<stage>` first";
- `RESULTS_DIR` can be pointed at a temp dir for tests. Artifacts are added to the registry in the phase that first writes them.

5. Pipeline entry point

New file `src/pipeline.py`: `python -m src.pipeline <stage> [options]` and `python -m src.pipeline status`. Phase 0 registers the `bench` and `train` stages; each later phase registers its own stage(s) here. `status` lists every stage, its artifacts and whether they exist.

6. Training rewrite (`src/network/train.py`, moved from `src/train.py`)

- **Single forward pass:** compute training accuracy from the same logits used for the loss.
- **In-memory dataset:**
- load CIFAR-10 once into tensors;
- do random-crop and flip augmentation as batched tensor ops on MPS;
- remove the DataLoader worker processes.
- **Validation split:** a fixed seeded 45,000 / 5,000 train/val split. Checkpoints are chosen by validation accuracy. The test set is used only in Phase 7 reporting.
- **Resume:** save model, optimizer and scheduler state each epoch to `checkpoints/v2/<arch>_seed<k>.pt` and resume if present; the epoch loop checks the deadline passed in by `run_jobs`.
- **Epoch count:** chosen by benchmark so that one float training is ≤ 12 min. The default stays 50 if it fits; otherwise the largest count in {30, 40, 50} that fits, with the same count for every architecture.

7. Network module (`src/network/arch.py`, replaces `src/model.py`)

- One architecture spec, `Arch(L, w)`, holds an ordered list of layer specs. It is the single definition of the network; nothing else describes layer shapes.
- `BASE_CHANNELS = [32, 64, 128, 256, 256]`. The fifth block repeats 256 so that L=5, w=1 stays near the flash limit instead of far above it.
- Allow `num_layers ∈ {2,3,4,5}`.

- `Arch.profile() -> list[dict]`: per layer, the type, C_in, C_out, H_in, W_in, H_out, W_out, MACs and weight count, computed from the layer specs without torch, so evaluation needs no model construction.
- `Arch.torch()` builds the `nn.Module` from the same layer specs. Phase 2 adds `Arch.onnx(b)` and Phase 4 adds quantization, both in `src/network/`.
- `src/bench_train.py` becomes the `bench` stage in `pipeline.py`.
- Delete `src/model.py` and `src/train.py` once moved.

Success Criteria

Automated Verification

- [ ] `pytest tests/test_network.py`: `Arch.profile()` MACs and parameter counts equal torchinfo on `Arch.torch()` for all 4 depths × 5 widths.
- [ ] `pytest tests/test_jobs.py` (fake clock, temp dir): finished jobs are skipped; the deadline stops between jobs; a job killed mid-write leaves no output; a rerun of a finished stage is a no-op.
- [ ] `pytest tests/test_results.py`: save/load round-trip; loading a missing artifact names its stage.
- [ ] `python -m src.pipeline bench` reports seconds per epoch for L=5, w=1.0 (the worst case) and the chosen epoch count.
- [ ] Resume test: kill training after 2 epochs, rerun, and confirm it continues from epoch 3.
- [ ] `python -m src.pipeline status` runs.

Manual Verification

- [ ] Laptop stays responsive during the benchmark.
- [ ] `results/v1/` holds every v1 file and `git status` shows renames, not deletions.

Pause for confirmation before Phase 1.

Phase 1: Hardware Cost Models (STM32F446RE)

Overview

Replace `energy_proxy.py` and the memory code with device-calibrated models. Constants in this phase are read from the STM32F446xC/E datasheet (DS10693) and the Cortex-M4 Technical Reference Manual, never invented. Cycle constants are placeholders until Phase 2 fits them.

All files in this phase go into `src/evaluator/`. They are internal to the evaluator module; from Phase 5 on, nothing outside `src/evaluator/` imports them.

Changes Required

0. Scenario

New file `src/evaluator/scenario.py`: a frozen dataclass `Scenario` holding the device, the workload (task set), kernel costs, idle mode (Sleep/Stop), the four limits and the loaded calibration. `Scenario.nominal()` builds the paper's main case. Every cost-model function takes its constants from a

`Scenario` (or the device inside it), never from module-level constants, so the Phase 5 sensitivity runs are new `Scenario` values and need no code changes.

1. Device constants

New file `src/evaluator/device.py`, one dataclass `STM32F446RE` with:

- **Memory:** flash 512 KB, SRAM 128 KB.
- **Operating points:** five (f, VOS scale, over-drive flag, I_run, I_sleep) entries taken from the datasheet's run-mode and sleep-mode current tables, using the "all peripherals disabled" columns. The five f values are the rows the tables actually list, spread from the lowest listed frequency at or above 30 MHz up to 180 MHz. Each constant carries a comment giving the table number and row.
- **Supply:** V_DD = 3.3 V.
- **Flash wait states per f,** from the reference manual's wait-state table.

2. Cycle model

New file `src/evaluator/cycles.py`:

```
def inference_cycles(profile, b, ws) -> int:
    # sum over layers: MACs_l * cpm[b][type_l] + weights_l * bytes(b) * k_flash(ws) + c_layer
    # plus c_fixed
```

- **Calibration:** `cpm[8]` and `cpm[32]` (cycles per MAC for int8 and fp32), `c_layer` and `c_fixed` are calibrated in Phase 2.
- **Literature-derived estimates** *(changed by the audit: TRM instruction counting has no journal precedent)*:
- `cpm[16] = cpm[8] · r16`, where r16 is the measured int16:int8 latency ratio on Cortex-M4F reported by Novac 2021 (Sensors, `10.3390/s21092984`).
- `cpm[4] = cpm[8] · r4`, where r4 is the measured int4:int8 kernel ratio reported for CMix-NN by Capotondi 2020 (IEEE TCAS-II, `10.1109/tcsii.2020.2983648`; Cortex-M7, so the core difference is stated).
- The table and page each ratio comes from go in the docstring; both are labelled estimates. A Cortex-M4 TRM instruction count is kept only as a cross-check and is not reported as a source.
- `k_flash(ws)` is added only if the Phase 2 leave-one-out int8 error exceeds 10 % without it.

3. Memory model

New file `src/evaluator/memory.py`:

- `flash_bytes(profile, b) = Σ weights·b/8 + scale/zero-point overhead (4 B per output channel for b ≤ 16) + code_allowance`
- `sram_bytes(profile, b) = max over layers (in_act + out_act + im2col_scratch) + RTOS_reserve (32 KB)`
- Activation bytes are 1 for b ∈ {4, 8}, 2 for b = 16 and 4 for b = 32.
- `code_allowance` and the scratch formula are calibrated against the Cloud report in Phase 2.

4. Energy model

New file `src/evaluator/energy.py`:

- Given (f point, active cycles per horizon, horizon length, number of inferences):
- `E_total = V_DD·I_run(f)·t_active + V_DD·I_sleep(f)·t_idle`
- `E_per_inference = E_total / N_inf`
- Active cycles include the periodic tasks, context switches and tick interrupts, all reported by the simulator in Phase 3.
- Delete `src/energy_proxy.py`.

Success Criteria

Automated Verification

- [ ] `pytest tests/test_evaluator.py` (cost-model section; these hand cases move to the `evaluate` interface in Phase 5):
- flash and SRAM match hand-computed values for 2 architectures;
- cycles are monotone in MACs for fixed b;
- energy at zero active cycles equals sleep energy;
- every operating-point constant is non-zero and f is strictly increasing.
- [ ] A race-to-idle sanity test prints energy per inference across the 5 frequencies for 3 architectures. The minimum need not sit at the lowest f, and the printout records where it does sit.

Manual Verification

- [ ] Spot-check 3 datasheet constants against the PDF table and row cited in comments.

Phase 2: Calibrate the Cycle and Memory Models on ST Silicon

Overview

Benchmark about 12 calibration architectures on real STM32 hardware through STM32Cube.AI Developer Cloud, then fit the Phase 1 placeholders.

Steps

1. **User task:** create a free ST account and log in to STM32Cube.AI Developer Cloud. In the board farm list, note the available Cortex-M4 F4 boards. Use an F446-family board if present, otherwise the closest F4 Cortex-M4 board. Record the board and its clock in `results/v2/cloud_board.json`.
2. **Export. New file** `src/network/export.py` adds `Arch.onnx(b)`. The `export-calib` stage in `pipeline.py` writes, through `run_jobs`, for 12 architectures (L ∈ {2,3,4,5} × w ∈ {0.25, 0.63, 1.0}, randomly initialized, since weights don't affect cycles): - an fp32 ONNX model; - an int8 static-quantized ONNX model (QDQ, per-channel weights) through `onnxruntime.quantization.quantize_static`, with 256 calibration images.

That is 24 files in `exports/calib/`, each under the Cloud's upload limit. 3. **Benchmark.** Upload each file and run the benchmark. Record measured inference cycles and time, flash and RAM in the `cloud_calibration` artifact (`results/v2/cloud_calibration.csv`, registered in `src/results.py`; Phase 7 reuses the same columns for `cloud_validation`). This is done in the browser, by the user or by me in the built-in browser while the user is logged in. 4. **Fit. New file** `src/evaluator/calibrate.py`, run as the `calibrate` stage, does a least-squares fit of `cpm[8]`, `cpm[32]`, `c_layer` and `c_fixed` (plus `k_flash` only if needed, see Phase 1) (and the memory `code_allowance` and scratch terms), then saves the `cycle_calibration` artifact (`results/v2/cycle_calibration.json`), which `Scenario.nominal()` loads. It reports the fit's R^2 and leave-one-architecture-out error. Delete v1 `src/collect_calibration.py`.

Success Criteria

Automated Verification

- [ ] `python -m src.pipeline calibrate` runs and writes the JSON; `Scenario.nominal()` loads it.
- [ ] Leave-one-out cycle error is printed for both int8 and fp32.

Manual Verification

- [ ] Leave-one-out cycle error ≤ 10% for int8. If it's larger, add per-layer-type terms (conv vs linear) and refit before moving on.
- [ ] The memory model's flash and RAM agree with the Cloud report within 10% on all 12 architectures.

Phase 3: RTOS Workload Simulator

Overview

Replace `src/scheduler_sim.py` with a deterministic, event-driven simulator of fixed-priority preemptive scheduling with round robin among equal priorities. It's written in plain Python, without SimPy, because SimPy preemption is awkward and a hand-written event loop is faster and easier to test.

Workload (fixed)

Task	Arrival	Work (cycles)	Priority
Sensor	period 10 ms	90,000	3 (highest)
Comms	period 50 ms	360,000	2
Inference	Poisson, mean 200 ms, 50 requests, ±5% jitter on cycles	`inference_cycles`	π: below = 1, equal = 2, above = 4

- **Kernel costs:** context-switch and 1 kHz tick costs are taken from FreeRTOS measurements on Cortex-M4 reported in Oliveira 2023 (Des Autom Embed Syst, `10.1007/s10617-023-09267-7`) or Ungurean 2020 (Symmetry, `10.3390/sym12040592`), converted to cycles; read the full text at the start of this phase. If neither reports a usable value, keep 200 and 100 cycles, label them assumptions, and rely on the ×0.5 / ×2 sensitivity in Phase 5. *(Audit: the 200/100 constants had no source.)*

- **Nominal task set:** the sensor/comms WCETs are a chosen nominal case (harmonic periods follow Pavić 2020, IEEE Access); Phase 5 sensitivity adds generated task sets.
- **Time slicing:** τ applies only among tasks of equal priority.
- **Deadlines:** periodic tasks have implicit deadlines equal to their period.
- **Horizon:** from 0 to the completion of the 50th inference.
- **Determinism:** the RNG is seeded from a fixed constant, so evaluations don't depend on the search method's seed.

Changes Required

New `src/evaluator/rtos_sim.py` (internal to the evaluator; delete v1 `src/scheduler_sim.py`). The task set and kernel costs above are stored in the `Scenario` workload, not as constants in the simulator. Its `simulate(inference_cycles, f_hz, tau_ms, pi, workload)` returns:

- `mean_latency_ms` and `p95_latency_ms` for inference;
- `miss_ratio` for the periodic tasks;
- `active_cycles` (all tasks, switches and ticks);
- `horizon_s`;
- `n_switches`.

Success Criteria

Automated Verification

- [ ] `pytest tests/test_rtos_sim.py` (an internal seam of the evaluator, tested directly because the hand cases need workloads other than the nominal one), with hand-computed cases:
- inference alone (no periodic tasks): latency equals cycles/f plus switch costs;
- inference above everything: periodic tasks miss when inference is longer than 10 ms;
- inference below: no periodic misses while utilization is below 1;
- equal priority with τ = 1 ms has more switches than with τ = 20 ms;
- π values give three distinct results on one mid-size design;
- the result is bit-identical across repeated calls.
- [ ] Timing: ≤ 30 ms per simulation for the worst design at 48–60 MHz, measured and printed.

Manual Verification

- [ ] A printed table of latency, miss ratio and energy for 3 designs × 3 π × 3 τ looks physically sensible: more priority means lower latency and more misses; a smaller slice means more switches.

Phase 4: Accuracy Data

Overview

Train the architectures, quantize at four precisions, and measure validation accuracy. This is the main compute phase: 6–7 chunks including the QAT pilot.

Changes Required

1. Quantization rewrite (`src/network/quantize.py`, replaces `src/quantization.py`)

b	Weights	Activations	Procedure
32	fp32	fp32	none
16	int16 per-channel symmetric	int16 per-tensor	post-training, 1,000 calibration images from the train split
8	int8 per-channel symmetric	int8 per-tensor	post-training, same calibration images
4	int4 per-channel symmetric	int8 per-tensor	QAT fine-tune from the float checkpoint, lr 1e-4, epoch count set by the QAT pilot below, then freeze

- Every Conv and Linear layer gets quantized, using `torch.ao.quantization` fake-quant modules so evaluation runs on MPS/CPU with the exact rounding.
- BatchNorm is folded into the convs before quantization.
- Delete v1 `src/quantization.py` once `src/network/quantize.py` is in place.
- **QAT pilot** *(audit: "5 epochs" had no source)*: on (2, 0.25) and (5, 1.0), run 4-bit QAT for up to 10 epochs and record validation accuracy per epoch. Use the smallest epoch count after which validation accuracy improves by < 0.2 points for both, and apply it to every architecture.

2. Training stage

The `accuracy` stage in `pipeline.py` builds the job list below and runs it through `run_jobs` (no separate queue file or driver script). The QAT pilot is the `qat-pilot` stage, run the same way:

- **Grid:** 20 architectures, L ∈ {2,3,4,5} × w ∈ {0.25, 0.4375, 0.625, 0.8125, 1.0}.
- **Seed noise:** (2, 0.25), (3, 0.625) and (5, 1.0) retrained with seeds 2 and 3.
- **Per job:** train float, then measure validation accuracy at b = 32, 16 and 8, then run 4-bit QAT and measure. Each job saves its four rows to the `accuracy_measurements` artifact (`results/v2/accuracy_measurements.csv`: arch, seed, b, val_acc, epochs, wall_s).
- **Deadline:** handled by `run_jobs`, which stops between jobs; training checks it between epochs.

3. Accuracy table

New file `src/evaluator/accuracy.py` (internal to the evaluator; `Scenario.nominal()` loads the table from the `accuracy_measurements` artifact, and tests can pass a small hand-made table):

- a table lookup of measured validation accuracy keyed by (L, w, b), as in tabular NAS benchmarks (NATS-Bench, Dong 2021, IEEE TPAMI, `10.1109/tpami.2021.3054824`); no interpolation *(audit: PCHIP had no journal precedent)*;
- `acc(L, w, b)` raises for a w that is not one of the 5 grid widths;
- delete `_fit_scaling_law`, `CALIBRATION_POINTS` and `src/validate_surrogate.py`; `src/objective.py` itself is deleted in Phase 5.

New file `src/report/accuracy_validation.py`, run as the `validate-accuracy` stage, saves the `accuracy_validation` artifact (`results/v2/accuracy_validation.json`):

- seed-noise standard deviation per repeated architecture and b (the table's uncertainty);

- the accuracy drop from b=32 to b=4 per architecture.

Success Criteria

Automated Verification

- [ ] Job queue complete: 26 float trainings (20 grid + 6 seed repeats), each with 4 precision measurements.
- [ ] `pytest tests/test_evaluator.py` (accuracy section): the lookup returns every measured value exactly and raises off-grid.
- [ ] `python -m src.pipeline validate-accuracy` runs and writes the JSON.

Manual Verification

- [ ] 4-bit accuracy is below 32-bit for most architectures. If 4-bit QAT turns out loss-free everywhere, report that honestly and keep the result; the cycle and memory models still separate the bit-widths.
- [ ] Seed noise is compared with the gaps between neighbouring widths and bit-widths; where a gap is within seed noise the paper says the two designs are not distinguishable.

Phase 5: Unified Evaluation and Reference Front

Changes Required

1. Search space (`src/search/space.py`)

- `SPACE`, the one definition of the design space; every method adapter in Phase 6 and the reference front read it, none re-declares it:
- `num_layers` {2,3,4,5}, ordered choice;
- `width_mult` ordered choice {0.25, 0.4375, 0.625, 0.8125, 1.0};
- `quant_bits` {4,8,16,32}, ordered choice;
- `time_slice` range [1, 20] ms;
- `priority` {0: below, 1: equal, 2: above}, unordered choice;
- `freq_idx` {0..4}, ordered choice into the device's operating points.
- Limits go in `Scenario.nominal()`, not config: latency 100 ms, miss ratio 0.01, flash 512 KiB, SRAM 128 KiB.
- Remove from `config/config.py` the search space, the SOO and grid constants, `SIM_VOLTAGE`, `SIM_CAPACITANCE` and `dvfs_factor`; config keeps only paths and compute settings.

2. Evaluator interface (`src/evaluator/__init__.py`, replaces `src/objective.py`)

- The evaluator's whole interface: `evaluate(design, scenario=Scenario.nominal()) -> Outcome`, plus `Scenario` and `Outcome`. Nothing else in `src/evaluator/` is imported from outside it.
- `Outcome` holds `accuracy, energy_uj, latency_ms, miss_ratio, flash_kb, sram_kb` and `acceptable`, which applies the scenario's four limits plus `accuracy ≥ 0.75` (the scoring floor, as in v1), and `violated`, the list of broken constraints (used for the Phase 6 rejection breakdown).

- **Simulation cache:** kept inside the evaluator, keyed by (scenario workload, cycles, f, τ, π). Callers never see it.
- **Memory short-circuit:** designs that break flash or SRAM skip the simulation and return `latency_ms = inf` and `miss_ratio = 1`. They still count as an evaluation for every method.
- Delete `src/objective.py`, `src/bo_experiment.py` and `src/baselines.py`.

3. Hypervolume reference corner

- Fixed at (acc 0.70, energy = the worst acceptable energy in the reference enumeration × 1.1, rounded up to a clean value).
- Chosen once in this phase, saved as the `hv_reference` artifact, never changed afterwards.
- A second corner (acc 0.75, energy = worst acceptable × 1.5) is also fixed now. HV results are reported at both, because method rankings can depend on the reference point (Pang 2024, IEEE TEVC, `10.1109/tevc.2023.3287399`).

4. Reference front

New file `src/search/reference_front.py` (delete v1 `src/reference_front.py`), run as the `front` stage. It calls only `evaluate(design, scenario)`:

- **Grid:** L (4) × w (5) × b (4) × f (5) × τ (10 values, log-spaced 1–20 ms) × π (3) = 12,000 designs. Exhaustive over every variable except τ, which is continuous in the search.
- **Pruning:** memory-infeasible (L, w, b) are dropped before simulation.
- **Caching:** handled by the evaluator's internal simulation cache.
- **Chunking:** one `run_jobs` job per (L, b) block, about 16 blocks.
- **Outputs:** the `reference_front` and `reference_front_summary` artifacts (`results/v2/reference_pareto_front.csv`, `reference_front_summary.json`: counts, hypervolume, wall time).
- **Sensitivity** (the `sensitivity` stage; each case below is one `Scenario` value derived from `Scenario.nominal()` and one `run_jobs` job, with no code changes): recompute the front under latency limits {50, 100, 200} ms × miss limits {0, 1, 5}%, plus idle in Stop mode instead of Sleep. This reuses the cached simulations, so there are no new runs. Added by the audit (these need new simulations, about 6 min each at 30 ms per simulation):
- kernel overheads ×0.5 and ×2;
- 2 generated task sets (HarmonicT method of Kim 2021, J Comput Sci Eng, `10.5626/jcse.2021.15.2.72`) at a low and a high periodic utilization, fixed seed.

Success Criteria

Automated Verification

- [ ] `pytest tests/test_evaluator.py`, through `evaluate` only: the Phase 1 cost-model hand cases, now checked on `Outcome`; memory-infeasible designs short-circuit; all six outputs are finite for acceptable designs; `evaluate` is deterministic; changing one `Scenario` field (e.g. kernel costs ×2) changes the outcome with no other code change.
- [ ] `grep` confirms no file outside `src/evaluator/` imports from its internal files.
- [ ] The enumeration completes and the summary JSON lists grid size, acceptable count and reference hypervolume.
- [ ] The sensitivity table is written.

Manual Verification

- [ ] Inspect the front's composition: which L, b, f and π values appear. Check that it is no longer pinned to a single b or L. Whatever it shows becomes the coupling paragraph in Results.

Phase 6: Search Methods and the 20-Seed Comparison

Overview

Six methods, the same `evaluate()`, 100 evaluations each, seeds 1–20 (1–30 if the pilot allows), one CSV per method and seed in `results/v2/runs/<method>/seed<k>.csv`, with a running hypervolume column.

Methods

Method	Package	Configuration
qNEHVI	Ax 1.3.1 / BoTorch	12 Sobol + GP; objectives acc↑ and energy↓; outcome constraints latency, miss ratio, flash, SRAM
qNParEGO	Ax / BoTorch	same initialization and constraints, ParEGO random scalarization
NSGA-II	pymoo 0.6.1.6	mixed-variable encoding, population 10 × 10 generations; constraints by constraint-domination
MOTPE	Optuna `TPESampler(multivariate=True)` multi-objective (Ozaki 2022, JAIR)	`constraints_func` with the four constraints (an Optuna option; no journal source, so no claim is made about it); 12 random startup trials
Random	NumPy	uniform over the mixed space
Sequential	Ax qNEHVI in two stages	stage 1: 50 evaluations over (L, w, b) with τ = 10 ms, π = equal, f = 180 MHz; stage 2: 50 evaluations over (τ, π, f, net), where net is a categorical index into the stage-1 front

Changes Required

- **New file** `src/search/harness.py`, interface `run_search(method, seed, budget=100, scenario=Scenario.nominal()) -> Trace`. The harness alone owns what must be identical across methods, so fairness is enforced in one place:
- the design space (`src/search/space.py`);
- the call to `evaluate`, and the count against the budget (memory-short-circuited designs count too);
- the rejection breakdown from `Outcome.violated`;
- the CSV columns and the running hypervolume at both reference corners;
- seeding of NumPy, torch and the method.
- **Method adapters** in `src/search/methods/` (`qnehvi.py`, `qnparego.py`, `nsga2.py`, `motpe.py`, `random.py`, `sequential.py`). Each implements only `ask() -> designs` and `tell(designs, outcomes)`, using the library's own ask/tell interface (Ax `get_next_trial`/`complete_trial`, pymoo `ask`/`tell`, Optuna `ask`/`tell`). The adapter translates `SPACE` into its library's encoding; it never calls `evaluate` or writes files. Sequential is an adapter that runs a stage-1 and a stage-2 qNEHVI adapter in turn.

- `compare` **stage** in `pipeline.py` (replaces the planned `run_comparison.py`): one `run_jobs` job per method × seed, saving the `run/<method>/<seed>` artifact (`results/v2/runs/<method>/seed<k>.csv`). `--method` and `--seeds` select a chunk.
- Delete `src/moo_experiment.py` and `src/robustness_check.py`. Move the hypervolume and IGD code from `src/pareto_metrics.py` into `src/search/metrics.py` and delete the v1 file.
- **Pilot:** 1 seed of qNEHVI, timed.
- If it takes more than 20 min per seed, merge flash and SRAM into one constraint, `mem_util = max(flash/limit, sram/limit) ≤ 1`, for all methods. This is one change in the harness's constraint list, not in each adapter.
- Rerun the pilot, then freeze the configuration before the sweep.
- **Seed count** *(audit: recent journal comparisons commonly use 30 runs)*: if qNEHVI takes ≤ 12 min per seed, run 30 seeds for every method; otherwise 20, with the minimum achievable Wilcoxon p reported next to each result.
- **New file** `src/search/stats.py`, run as the `stats` stage, saves the `comparison_summary` artifact (`results/v2/comparison_summary.json`):
- HV % of reference (at both reference corners) and IGD at 25, 50 and 100 evaluations (median and IQR);
- evaluations needed to reach 90% of the reference hypervolume;
- a Friedman test across the 6 methods;
- qNEHVI against each other method: one-sided Wilcoxon signed-rank with Holm correction, plus Cliff's δ;
- rejected-evaluation breakdown by constraint;
- all p-values reported together with the test's minimum achievable value.

Success Criteria

Automated Verification

- [ ] 120 (or 180) CSVs, one per method × seed, each with 100 rows.
- [ ] `pytest tests/test_search.py`: a fake method adapter (proposes fixed designs) confirms the harness stops at exactly the budget, counts short-circuited designs, writes the fixed column set, and produces a non-decreasing HV trace; each real adapter completes a 5-evaluation smoke run.
- [ ] `python -m src.pipeline stats` runs and writes the summary; HV traces are non-decreasing.
- [ ] Re-running a finished chunk is a no-op.

Manual Verification

- [ ] The results are reported whichever way they come out. If qNEHVI does not beat a baseline, the paper says so. If Sequential matches Joint, that becomes the finding and the framing changes. Writing waits for these numbers.

Phase 7: Held-Out Hardware Validation and Test-Set Reporting

Steps

1. Select 10 designs: - 6 from the median-seed qNEHVI front, spread along the front; - 4 off-front, chosen to cover the L × b combinations not yet represented.

Only b ∈ {8, 32} can run on the Cloud. Designs at b ∈ {4, 16} are benchmarked at int8 and the int4/int16 adjustment is reported as model-only. 2. Train or reuse the weights (the `train` stage), export with `Arch.onnx(b)` (the `export-validate` stage), benchmark on the same Cloud board, and record results in the `cloud_validation` artifact (`results/v2/cloud_validation.csv`, same columns as `cloud_calibration`). 3. **New file** `src/report/hw_validation.py`, run as the `validate-hw` stage: predicted (through `evaluate`) vs measured cycles, flash and RAM for these designs, which were not used in calibration. Reports MAE % and max %. 4. Evaluate test-set accuracy once for the 6 front designs, at their b, and report it next to the predicted validation accuracy (the `test-accuracy` stage; the only code path that reads the CIFAR-10 test split).

Success Criteria

- [ ] The `hw_validation` and `test_accuracy` artifacts (`results/v2/hw_validation.json`, `results/v2/test_accuracy.json`) are written.
- [ ] The errors are reported as measured, whatever they turn out to be.

Phase 8: Figures and Tables

All figures grayscale (`black!N` in TikZ, gray levels and marker shapes in matplotlib), PDF output, in `figures/v2/`.

1. `paper/figures/fig_architecture.tex`, redrawn: design vector → layer profile → {accuracy model, memory model, cycle model} → workload simulator → energy model → constraint check → objectives.
2. `fig_front.pdf`: the reference front as a line, plus each method's median-seed front, with distinct markers per method.
3. `fig_convergence.pdf`: HV % against evaluations, median line and IQR band for each of the six methods.
4. Cycle validation, predicted against measured with a y = x line: a figure if there's space at the Phase 9 fit, otherwise a small table.
5. Tables: - design space; - workload and device constants; - accuracy table summary (seed noise, 32→4-bit drop); - method comparison (HV %, IGD, evaluations to 90%, p, δ); - hardware validation.

All tables use the "Table N:" bordered style with shaded headers. 6. Delete `paper/figures/fig_search.tex` and its PDF.

Success Criteria

- [ ] `src/report/figures.py` and `src/report/tables.py` (replacing `src/plot_results.py`, which is deleted) regenerate every figure and table from the results store with one command, `python -m src.pipeline report`.
- [ ] Cleanup check: `src/` contains only the Code Layout files; no v1 module remains (`baselines`, `bo_experiment`, `collect_calibration`, `energy_proxy`, `model`, `moo_experiment`, `objective`, `pareto_metrics`, `plot_results`, `quantization`, `reference_front`, `robustness_check`, `scheduler_sim`, `train`, `validate_surrogate` are all deleted or moved).
- [ ] Figures inspected at 300 DPI: legible at column width and grayscale-distinguishable.

Phase 9: Paper Rewrite and Verification

Page plan (7 pages, IEEEtran conference)

Section	≈ Pages	Content
Abstract + Intro + related work	0.9	problem, gap (no RTOS parameters in hardware-aware NAS), contributions
Method	2.4	design space, workload, device models with calibration, accuracy data, constraints, search
Results	2.8	reference front and coupling, method comparison, Sequential ablation, hardware validation, sensitivity, limitations
Conclusion	0.3	
References	0.6	about 30–35 journal articles

Changes Required

- Rewrite `paper/main.tex`, `intro.tex`, `methodology.tex`, `results.tex` and `conclusion.tex` from the v2 results.
- `refs.bib`:
- every entry must be a journal article with a print year of 2020 or later;
- replace the 17 pre-2020 entries using the mapping in `docs/plans/2026-10-11-v2-literature-audit.md` §2;
- add the sources listed in the audit table for each claim the text makes (all already verified on Crossref);
- every new entry verified on `api.crossref.org/works/<DOI>`, using the print year;
- remove entries no longer cited.
- Name the datasheet (DS10693), the Cortex-M4 TRM, STM32Cube.AI Developer Cloud, Ax, BoTorch, pymoo and Optuna in the text only.
- **New file** `src/report/paper_numbers.py`, run as the `paper-numbers` stage, reads only through the results store and saves the `paper_numbers` artifact (`results/v2/paper_numbers.json`).
- **New file** `scripts/check_paper_numbers.py` extracts the numbers from the `.tex` files and flags any not found in the JSON, up to rounding.

Success Criteria

Automated Verification

- [ ] `cd paper && tectonic -X compile main.tex --outdir build_preview` succeeds with exactly 7 pages.
- [ ] A banned-word grep over `paper/*.tex` returns zero hits.
- [ ] Pattern counts: "rather than" ≤ 2, ", so" ≤ 6 per file, zero spaced em dashes.
- [ ] `scripts/check_paper_numbers.py` reports zero unmatched numbers.
- [ ] Every `refs.bib` entry is `@article`, cited, and has `year ≥ 2020` (script check).

Manual Verification

- [ ] Page-by-page render at 150 DPI shows no blank space and page 7 is filled.

- [ ] Read-through for claims stronger than the evidence, especially around the 4-bit cycle estimates and the energy model, which is not measured.

Testing Strategy

Unit Tests (`tests/`)

One test file per deep module, each through that module's interface:

- `test_jobs.py`: skip-done, deadline, atomic writes, no-op rerun (fake clock, temp dir).
- `test_results.py`: save/load round-trip, missing-artifact error.
- `test_network.py`: `Arch.profile()` against torchinfo on `Arch.torch()`.
- `test_evaluator.py`: memory, cycle and energy hand cases, accuracy lookup exactness and off-grid error, short-circuit, determinism, finiteness, `Scenario` substitution, all through `evaluate`.
- `test_rtos_sim.py`: the hand-computed scheduling cases from Phase 3 (the evaluator's one tested internal seam).
- `test_search.py`: harness budget, columns and HV trace with a fake adapter; smoke run of each real adapter.

Validation Studies (reported in the paper)

- Cycle and memory models: calibration fit (Phase 2) and held-out silicon check (Phase 7).
- Accuracy table: seed noise per architecture and bit-width (Phase 4).
- Thresholds, idle mode, kernel overheads and task sets: reference-front sensitivity (Phase 5).

Manual Steps

1. After each phase, review the printed sanity tables before continuing.
2. Before Phase 9, review `comparison_summary.json` together and agree on the headline claim.

References

- v1 handoff: `~/.claude/handoffs/bayesian-optimization-microprocessor/general/2026-10-10_12-05-23_paper-audit-journal-refs.md`
- v1 MOO upgrade plan: `docs/plans/2026-08-26-multiobjective-bo-upgrade.md`
- Style rules: memory `feedback_paper_writing_style.md`
- Literature audit (journal backing, print year ≥ 2020): `docs/plans/2026-10-11-v2-literature-audit.md`
- Code being replaced: every v1 file in `src/` (see the Phase 8 cleanup check); the phase that replaces each file deletes it.
- Architecture review (2026-10-11): five deep modules behind one pipeline entry point, folded into Phases 0–9 (see Code Layout).